Version control & Git

A guide for people who already use Bash, Python, and R

Friederike Wölke

2026-10-17

THE AMERICAN STATISTICIAN
2018, VOL. 72, NO. 1, 20–27
https://doi.org/10.1080/00031305.2017.1399928

Taylor & Francis
Taylor & Francis Group

Check for updates

Excuse Me, Do You Have a Moment to Talk About Version Control?

Jennifer Bryan

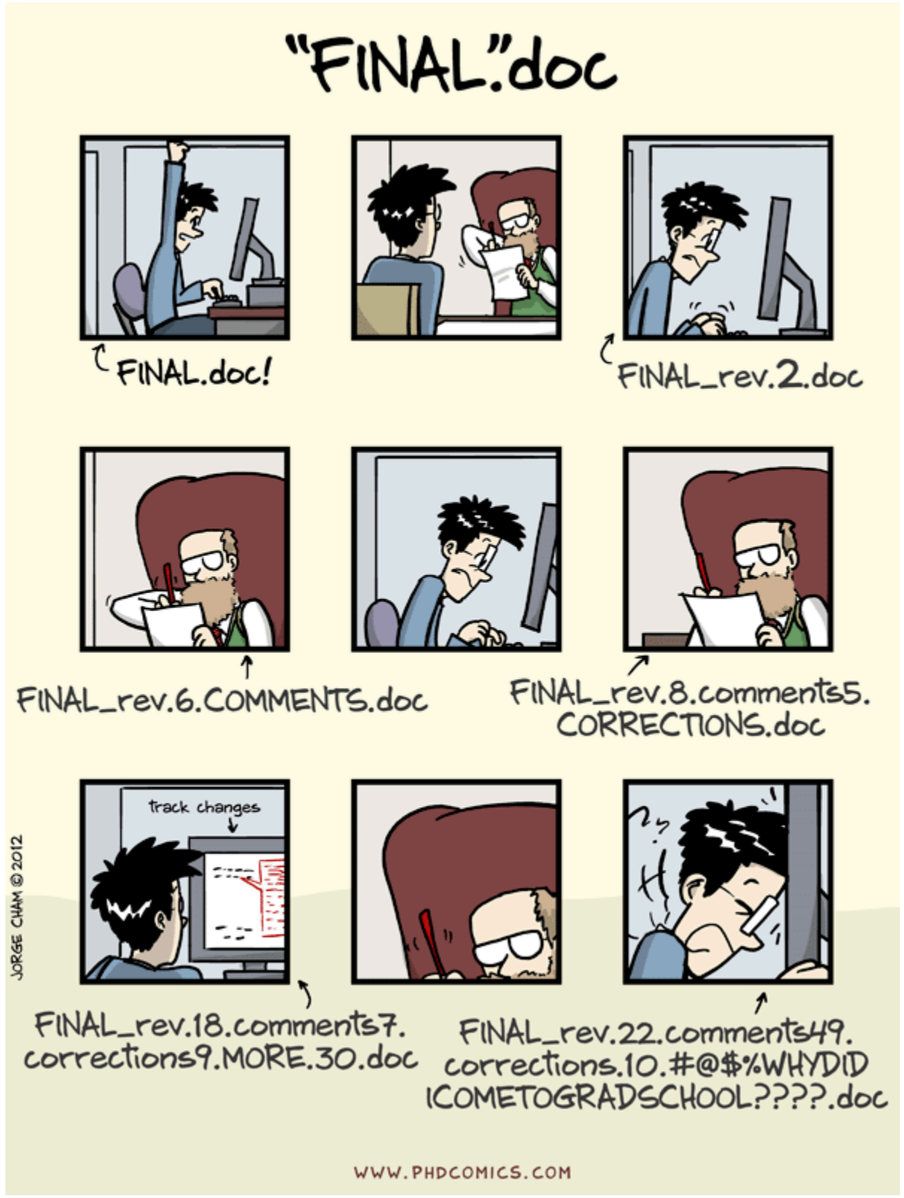
RStudio and the Department of Statistics, University of British Columbia, Vancouver, Canada

Learning objectives

- understand the benefits of version control
- know where to find help when setting up Git and GitHub on your computer
- understand the concept of: `stage` (`add`), `commit`, `push`, `pull`, `clone`
- work with GitHub repositories from the command line, RStudio, Visual Studio Code, Positron... (wherever you do your coding)

Version control - What is it?

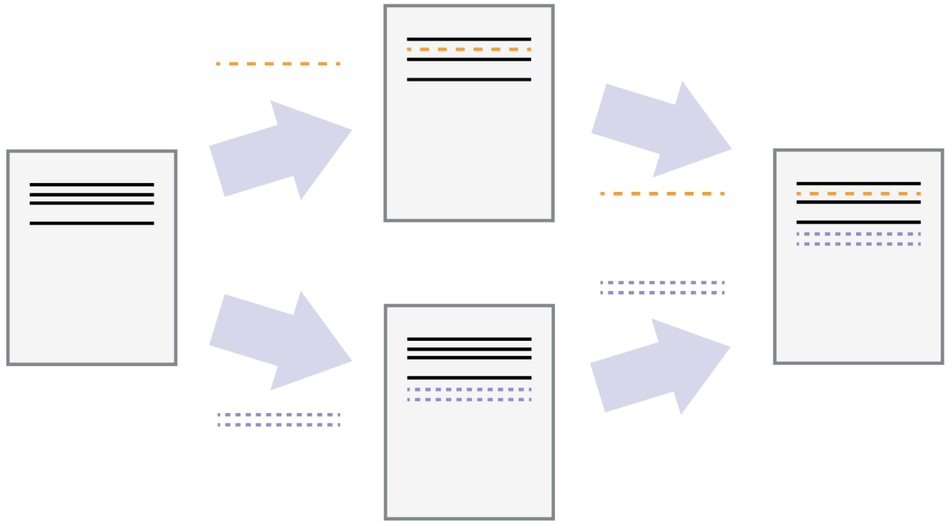
(Sources: PhD Comics “Final.doc”, The Carpentries 2025)



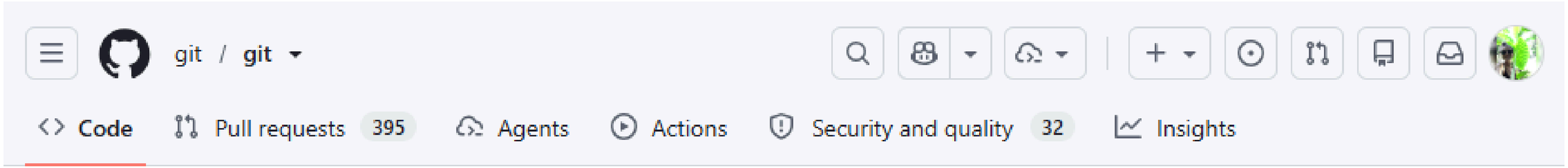
A



B



What is git?



Commit e83c516

Browse files

Linus Torvalds committed on Apr 8, 2005

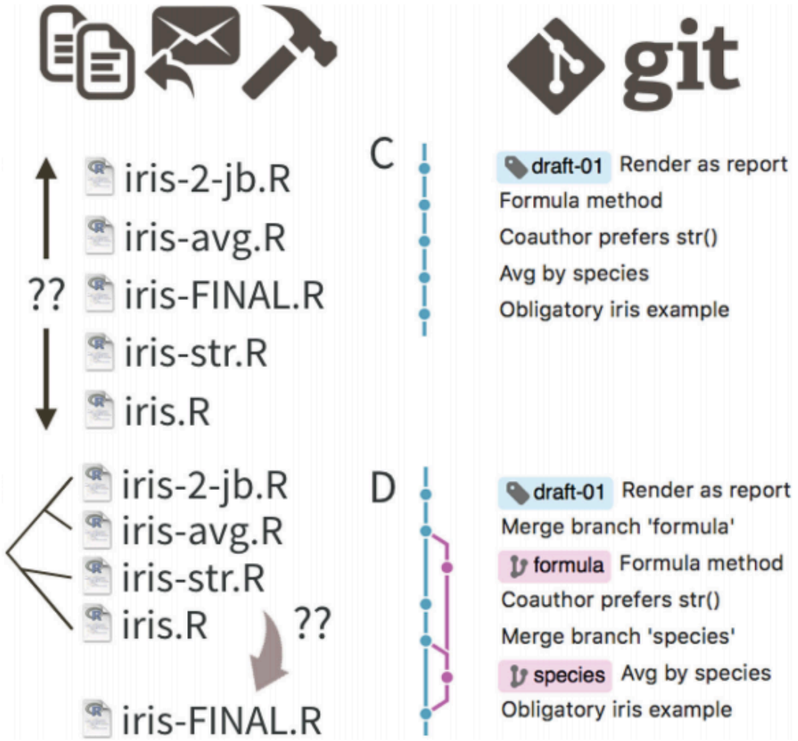
```
Initial revision of "git", the information manager from hell
```

master · v2.56.0 ··· v0.99 0 parents commit e83c516

11 files changed +1,244

- fast
- optimal for non-linear workflows
- strong safeguards against file corruption

We all know the problem



(Bryan 2018)

The moment when two (or more) coauthors send you their own version of the manuscript with tracked changes and you have to figure out how to combine it all ...

???

Git: three main concepts

1. **Snapshot**: save a meaningful state of your project.
2. **Compare**: see exactly what changed, and when.
3. **Recover**: return to a known-good state when an experiment goes sideways.

Git is not a backup folder. It is a timeline of deliberate project snapshots.

Vocabulary[1]

`Version Control` A system for maintaining current and past copies of all of your files and a record of all the changes you have made;

`git` Version control software for tracking your project on your local computer

`GitHub` A free, public remote website where your project is permanently hosted

`Repository` Any collection of files that belong to the same project, plus a few special git files that set up the version control. Also called a repo by hipster programmers.

`Clone` Create a local copy of a repository from the GitHub website to your own website. You can clone from your own repository or anyone else's that is posted on GitHub. Once the files are on your local computer you can edit or use them as you like. However, you can only change files on GitHub for the repositories that you yourself created or for projects that you have been invited to collaborate on with others.

`Commit (the noun)` A snapshot of your file system at a particular time. Since your last commit, it keeps track of the files you currently have, the ones you have deleted or added, and any changes you have made to files. Each commit has a unique identifier ("SHA")

`Commit (the verb)` When you commit, you are taking a snapshot of your files at the current time. You must add a summary of the commit that briefly describes what changes you have made (such as "add new homework file"). Note that all commits affect only the local copy of your repository. They do not affect the remote copy that is stored on GitHub

`Push` Pushing a set of changes means copying them from your local repository up to GitHub.

`Pull(=fetch)` Pulling means copying the repository in its current state on GitHub down to your local repository. This would incorporate any changes that others have made (and pushed) in a collaborative project.

Git is language-agnostic

Git does not care whether the file is Python, R, Bash, Markdown, or a dataset.

```
project/
├── analysis.py
├── model.R
├── run.sh
├── report.qmd
└── data/
    └── observations.csv
```

Git tracks **files and their history**. Your Python environment, R packages, and shell are separate concerns that we can record alongside the code (e.g., in R using `renv` lockfiles or in python using `requirements.txt` files)

Three places where a change can live

Place	Meaning	Git command
Working folder	Your files right now. You can edit, run, and inspect them.	`git status` , `git diff`
Staging area	A review basket: the exact changes you want in the next snapshot.	`git add`
Repository	The saved timeline of snapshots, called commits.	`git commit`

Git or GitHub?

Git



- local software (offline)
- tracks file changes
- needs to be installed (it’s free)
- works natively in the terminal

GitHub



- Remote code hosting and collaboration platform (owned by Microsoft)
- Common alternatives: Bitbucket and GitLab (some instances are self-hosted by universities)
- Additional features include collaboration, project management, publishing, and Actions
- Free account required
- Works via the web, terminal, and GUI clients (such as GitHub Desktop)

Benefits[1] of web-hosting on GitHub, GitLab, Bitbucket, or similar

- Organizing, recording, and disseminating your work - exposure prevents neglecting these tasks
- Structured collaboration and access to powerful tools for asynchronous work
- Creating web sites in 60 seconds
- Can be turned into citable content (add `CITATION.txt` file or connect to Zenodo and get a DOI)
- Acts as shareable lab notebook for computational work

Setting up git and/or GitHub

This step is always very painful. **Brace yourself!!**

I've collated information from multiple sources[1] to make it as painless as possible.

You will be able to see and download the setup instructions as a .qmd file from here.

Workflows in Git(Hub)

After initializing a folder as a git repository (`git init` or `git clone`)

1. Create files, make changes
2. `git status`
3. stage changes: `git add -A`
4. commit to make a snapshot: `git commit -am "Commit Message"`
   Do this frequently!
5. `git status`
6. Upload commits to GitHub: `git push`
7. Download latest version from GitHub: `git pull`
8. `git status`

repeat #1-8

Which files to track?

Depends on your aims. Usually:

Track:

- source files (files that are created and edited by hand: scripts, raw data, metadata)
- derived products with external value (e.g., figures, tables, reports)
- configuration files (e.g., .Rproj, .gitignore)

Ignore:

- intermediates (e.g. log files, temporary files generated from source files)

Prerequisites for happy ‘gitting’

- have git installed
- have your credentials configured on your local machine
- have a GUI that you like and feel comfortable with
  - I like the built-in git GUI in Visual Studio Code but also use GitHub Desktop and the RStudio-panel
- establish a routine (`commit` frequently, `push` often)
- if you switch between different machines or collaborate with others *always* start with synchronizing the changes to your current machine `pull` or `fetch`

Establish a routine when working solo

- use *projects* (R projects, Python projects, Positron folders/projects, VSC workspaces), i.e., **dedicate a specific folder to each project**
- initialize *local folders* as git repositories
- initialize *new folders* for future projects directly as git repositories
- *stage* changes that you want to keep
- *commit* them to the repository and write a summary what was changed (**annotation!!**)
- split *commits* thematically into smaller bits
- big change: create a *branch*
  - if you are happy: `merge` the branch into your `main branch`
  - if you are not happy: delete, archive, or ignore and switch back to `main`
- after a *milestone* is reached or at the end of the day: `push`
- when working on *different machines* or editing files via the *web browser*: `pull` often to avoid critical errors
- use issues on GitHub to track your own to-dos for your project

References

Bryan, Jennifer. 2018. “Excuse Me, Do You Have a Moment to Talk about Version Control?” *The American Statistician* 72 (1): 20–27. https://doi.org/10.1080/00031305.2017.1399928.

Bryan, Jennifer, and Jim Hester. 2024. “Happy Git and GitHub for the useR.” 2024. https://happygitwithr.com/.

Gotelli, Nicholas J. 2025. “GitHub.” https://gotellilab.github.io/Bio381/Lectures/GitHub.html.

Naturhistoriska riksmuseet, Johan Nylander. 2026. “Git-Github-Intro: A Brief Introduction to Git and GitHub.” GitHub. https://github.com/Naturhistoriska/git-github-intro.

Posit Software, PBC. 2026. “Version Control – RStudio User Guide.” 2026. https://docs.posit.co/ide/user/ide/guide/tools/version-control.html.

Rodrigues, Bruno. 2024. “Version Control with Git.” 2024. https://raps-with-r.dev/git.html.

SSoQE; Ondřej Mottl. 2025. “SSoQE-Version_control: Presentation and Materials for Lecture ‘Version Control’.” GitHub. https://github.com/SSoQE/SSoQE-Version_Control.

The Carpentries. 2025. “Version Control with Git.” 2025. https://swcarpentry.github.io/git-novice/.